HyperV.Tests - Dokumentacja Testów API

Przegląd

Ten projekt zawiera kompleksowy zestaw testów dla HyperV Agent API. Testy są zorganizowane w kilka kategorii, aby zapewnić pełne pokrycie funkcjonalności API.

Struktura Projektu

```
HyperV.Tests/
├── Controllers/           # Testy jednostkowe dla kontrolerów
│   ├── ServiceControllerTests.cs      # ☑ Ukończone - 7 testów
│   ├── JobsControllerTests.cs         # ☑ Ukończone - 19 testów
│   ├── VmsControllerTests.cs          # 📋 Template do implementacji
│   ├── ContainersControllerTests.cs   # 📋 Template do implementacji
│   ├── StorageControllerTests.cs      # 📋 Template do implementacji
│   └── NetworksControllerTests.cs     # 📋 Template do implementacji
├── EndToEnd/              # Testy end-to-end z rzeczywistym serwerem
│   ├── EndToEndTestBase.cs            # ☑ Klasa bazowa
│   └── BasicEndpointsE2ETests.cs      # ☑ Podstawowe testy E2E
├── Integration/           # Testy integracyjne (zarezerwowane)
├── EdgeCases/             # Testy przypadków brzegowych (zarezerwowane)
├── Performance/           # Testy wydajnościowe (zarezerwowane)
└── Helpers/               # Klasy pomocnicze
    ├── BaseTestClass.cs               # ☑ Klasa bazowa dla testów
jednostkowych
    ├── TestWebApplicationFactory.cs   # ☑ Factory dla testów
    ├── TestDataGenerator.cs           # ☑ Generator danych testowych
    ├── ApiResponseVerifier.cs         # ☑ Weryfikator odpowiedzi API
    └── ServiceMocks.cs                # ☑ Mocki serwisów
```

Status Implementacji

☑ Ukończone (26 testów)

- **ServiceController**: 7 testów
  - GET /api/v1/service/health
  - GET /api/v1/service/info
  - Weryfikacja struktur odpowiedzi
  - Testowanie kodów statusu
- **JobsController**: 19 testów
  - GET /api/v1/jobs/storage
  - GET /api/v1/jobs/storage/{jobId}
  - GET /api/v1/jobs/storage/{jobId}/affected-elements

    - POST /api/v1/jobs/storage/{jobId}/cancel
    - DELETE /api/v1/jobs/storage/{jobId}
    - Testy przypadków błędów (404, 500)
    - Weryfikacja wywołań serwisów

🔄 W trakcie implementacji

- **EndToEndTestBase**: Klasa bazowa dla testów E2E
- **BasicEndpointsE2ETests**: Podstawowe testy end-to-end

📋 Do implementacji

- **VmsController**: ~15-20 testów (wszystkie endpointy VM)
- **ContainersController**: ~10 testów (wszystkie endpointy kontenerów)
- **StorageController**: ~20 testów (wszystkie endpointy pamięci masowej)
- **NetworksController**: ~6 testów (wszystkie endpointy sieciowe)

Testowane API Endpoints

Service Management

- ☑ `GET /api/v1/service/health` - Status agenta
- ☑ `GET /api/v1/service/info` - Informacje o agencie

Job Management

- ☑ `GET /api/v1/jobs/storage` - Lista zadań pamięci masowej
- ☑ `GET /api/v1/jobs/storage/{jobId}` - Szczegóły zadania
- ☑ `GET /api/v1/jobs/storage/{jobId}/affected-elements` - Elementy których dotyczy zadanie
- ☑ `POST /api/v1/jobs/storage/{jobId}/cancel` - Anulowanie zadania
- ☑ `DELETE /api/v1/jobs/storage/{jobId}` - Usunięcie zadania

Virtual Machines (Do implementacji)

- 📋 `GET /api/v1/vms` - Lista maszyn wirtualnych
- 📋 `POST /api/v1/vms` - Tworzenie maszyny wirtualnej
- 📋 `GET /api/v1/vms/{name}/present` - Sprawdzanie obecności VM
- 📋 `GET /api/v1/vms/{name}/properties` - Właściwości VM
- 📋 `POST /api/v1/vms/{name}/start` - Uruchomienie VM
- 📋 `POST /api/v1/vms/{name}/stop` - Zatrzymanie VM
- 📋 `POST /api/v1/vms/{name}/shutdown` - Wyłączenie VM
- 📋 `POST /api/v1/vms/{name}/terminate` - Terminacja VM
- 📋 `POST /api/v1/vms/{name}/pause` - Wstrzymanie VM
- 📋 `POST /api/v1/vms/{name}/resume` - Wznowienie VM
- 📋 `POST /api/v1/vms/{name}/save` - Zapisanie stanu VM
- 📋 `POST /api/v1/vms/{name}/configure` - Konfiguracja VM
- 📋 Endpointy związane ze snapshotami VM

- 📋 Endpointy związane z pamięcią masową VM

Containers (Do implementacji)

- 📋 `POST /api/v1/containers` - Tworzenie kontenera
- 📋 `GET /api/v1/containers` - Lista kontenerów
- 📋 `GET /api/v1/containers/{id}` - Szczegóły kontenera
- 📋 `POST /api/v1/containers/{id}/start` - Uruchomienie kontenera
- 📋 `POST /api/v1/containers/{id}/stop` - Zatrzymanie kontenera
- 📋 `POST /api/v1/containers/{id}/terminate` - Terminacja kontenera
- 📋 `POST /api/v1/containers/{id}/pause` - Wstrzymanie kontenera
- 📋 `POST /api/v1/containers/{id}/resume` - Wznowienie kontenera
- 📋 `DELETE /api/v1/containers/{id}` - Usunięcie kontenera

Storage Management (Do implementacji)

- 📋 `POST /api/storage/vhd` - Tworzenie VHD
- 📋 `PUT /api/storage/vhd/attach` - Podłączanie VHD
- 📋 `PUT /api/storage/vhd/detach` - Odłączanie VHD
- 📋 `PUT /api/storage/vhd/resize` - Zmiana rozmiaru VHD
- 📋 `GET /api/storage/vhd/metadata` - Metadane VHD
- 📋 `PUT /api/storage/vhd/metadata` - Aktualizacja metadanych VHD
- 📋 Wszystkie inne endpointy StorageController

Network Management (Do implementacji)

- 📋 `POST /api/v1/networks/nat` - Tworzenie sieci NAT
- 📋 `DELETE /api/v1/networks/{id}` - Usunięcie sieci
- 📋 `GET /api/v1/networks/{id}/properties` - Właściwości sieci
- 📋 `POST /api/v1/networks/{networkId}/endpoints` - Tworzenie endpointu
- 📋 `DELETE /api/v1/networks/endpoints/{endpointId}` - Usunięcie endpointu
- 📋 `GET /api/v1/networks/endpoints/{endpointId}/properties` - Właściwości endpointu

Uruchamianie Testów

Testy Jednostkowe

```
# Wszystkie testy jednostkowe
dotnet test src/HyperV.Tests/HyperV.Tests.csproj

# Testy konkretnego kontrolera
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter
"ServiceControllerTests"
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter "JobsControllerTests"

# Testy z szczegółowym logowaniem
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --logger
"console;verbosity=detailed"
```

Testy End-to-End

```
# UWAGA: Wymagają uruchomionego serwera API!

# 1. Uruchom serwer API w osobnym terminalu
cd src/HyperV.Agent
dotnet run

# 2. W innym terminalu uruchom testy E2E
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter "E2ETests"
```

Klasy Pomocnicze

TestDataGenerator

Generuje przykładowe dane testowe dla wszystkich typów żądań:

```
var vmRequest = TestDataGenerator.CreateVmRequest("test-vm", "TestVM");
var containerRequest = TestDataGenerator.CreateContainerRequest("test-
container");
var vhdRequest = TestDataGenerator.CreateVhdRequest();
```

ApiResponseVerifier

Weryfikuje odpowiedzi API:

```
var content = await ApiResponseVerifier.VerifySuccessResponse(response);
var json = ApiResponseVerifier.VerifyJsonResponse(content);
ApiResponseVerifier.VerifyHealthResponse(json);
```

ServiceMocks

Konfiguruje mocki serwisów dla testów jednostkowych:

```
var mockJobService = ServiceMocks.ConfigureJobServiceMock();
var mockStorageService = ServiceMocks.ConfigureStorageServiceMock();
```

Rozszerzanie Testów

Dodawanie Nowego Kontrolera

1. **Stwórz plik testowy**:

```
// src/HyperV.Tests/Controllers/NewControllerTests.cs
public class NewControllerTests
{
    private readonly NewController _controller;
    private readonly Mock<INewService> _mockService;

    public NewControllerTests()
    {
        _mockService = new Mock<INewService>();
        _controller = new NewController(_mockService.Object);
    }

    [Fact]
    public async Task GetSomething_ShouldReturnOkResult()
    {
        // Test implementation
    }
}
```

2. **Dodaj mocki do ServiceMocks.cs**:

```
public static Mock<INewService> ConfigureNewServiceMock()
{
    var mock = new Mock<INewService>();
    // Configure mock behavior
    return mock;
}
```

3. **Dodaj generator danych do TestDataGenerator.cs**:

```
public static NewRequest CreateNewRequest(params)
{
    return new NewRequest { /* properties */ };
}
```

Dodawanie Testów End-to-End

1. **Dziedzicz z EndToEndTestBase**:

```
public class NewFeatureE2ETests : EndToEndTestBase
{
    [Fact]
    public async Task NewFeature_E2E_ShouldWork()
```

```
        {
            // Test real API calls
            var response = await GetAsync("/api/v1/new-feature");
            AssertStatusCode(response, HttpStatusCode.OK);
        }
    }
```

Wzorce Testowe

Test Jednostkowy z Mockami

```
[Fact]
public async Task Method_WithValidInput_ShouldReturnExpectedResult()
{
    // Arrange
    var input = TestDataGenerator.CreateValidInput();
    _mockService.Setup(x => x.DoSomething(input))
        .ReturnsAsync(expectedResult);

    // Act
    var result = await _controller.Method(input);

    // Assert
    result.Should().BeOfType<OkObjectResult>();
    _mockService.Verify(x => x.DoSomething(input), Times.Once);
}
```

Test End-to-End

```
[Fact]
public async Task Feature_E2E_CompleteWorkflow()
{
    // Arrange
    var createRequest = TestDataGenerator.CreateRequest();

    // Act & Assert
    // 1. Create resource
    var createResponse = await PostAsync("/api/v1/resource", createRequest);
    AssertStatusCode(createResponse, HttpStatusCode.OK);

    // 2. Verify resource exists
    var getResponse = await GetAsync("/api/v1/resource/test-id");
    AssertStatusCode(getResponse, HttpStatusCode.OK);

    // 3. Delete resource (cleanup)
    var deleteResponse = await DeleteAsync("/api/v1/resource/test-id");
    AssertStatusCode(deleteResponse, HttpStatusCode.OK);
}
```

Pokrycie API

Aktualnie Przetestowane (26 testów)

- ☑ **ServiceController**: 100% pokrycie (2/2 endpointy)
- ☑ **JobsController**: 100% pokrycie (5/5 endpointów)

Do Implementacji (~50+ testów)

- 📋 **VmsController**: 0% pokrycie (0/15+ endpointów)
- 📋 **ContainersController**: 0% pokrycie (0/9 endpointów)
- 📋 **StorageController**: 0% pokrycie (0/20+ endpointów)
- 📋 **NetworksController**: 0% pokrycie (0/6 endpointów)

Kategorie Testów

1. Testy Jednostkowe (`Controllers/`)

- Testują logikę kontrolerów z zamockowanymi zależnościami
- Szybkie wykonanie, nie wymagają zewnętrznych zasobów
- Fokus na logice biznesowej i obsłudze błędów

2. Testy End-to-End (`EndToEnd/`)

- Testują rzeczywiste wywołania API na działającym serwerze
- Wymagają uruchomionego serwera HyperV.Agent
- Testują pełne scenariusze użycia

3. Testy Integracyjne (`Integration/`) - Zarezerwowane

- Testują integrację między komponentami
- Mogą używać rzeczywistych serwisów, ale kontrolowanego środowiska

4. Testy Przypadków Brzegowych (`EdgeCases/`) - Zarezerwowane

- Testują nietypowe scenariusze i obsługę błędów
- Nieprawidłowe dane wejściowe, błędy sieci itp.

5. Testy Wydajnościowe (`Performance/`) - Zarezerwowane

- Testują wydajność kluczowych operacji
- Pomiary czasu odpowiedzi, przepustowości

Instrukcje Uruchamiania

Przygotowanie Środowiska

1. **Upewnij się, że wszystkie zależności są zainstalowane**:

```
dotnet restore src/HyperV.Tests/HyperV.Tests.csproj
```

2. **Zbuduj projekt**:

```
dotnet build src/HyperV.Tests/HyperV.Tests.csproj
```

Testy Jednostkowe

```
# Uruchom wszystkie testy jednostkowe
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter
"TestCategory!=EndToEnd"

# Uruchom z raportowaniem pokrycia kodu
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --collect:"XPlat Code
Coverage"
```

Testy End-to-End

1. **Uruchom serwer API**:

```
cd src/HyperV.Agent
dotnet run
```

2. **W osobnym terminalu uruchom testy E2E**:

```
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter
"TestCategory=EndToEnd"

# Lub konkretną klasę testową
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter
"BasicEndpointsE2ETests"
```

Generowanie Raportów

```
# Raport HTML z wynikami testów
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --logger html --results-
directory TestResults

# Raport XML dla CI/CD
```

```
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --logger trx --results-directory TestResults
```

Najlepsze Praktyki

1. Nazewnictwo Testów

- `MethodName_Scenario_ExpectedResult`
- Przykład: `GetHealth_ShouldReturnHealthyStatus`

2. Struktura Testów (AAA)

```
[Fact]
public async Task Method_Scenario_ExpectedResult()
{
    // Arrange - przygotowanie danych testowych
    var input = TestDataGenerator.CreateValidInput();

    // Act - wykonanie testowanej operacji
    var result = await _controller.Method(input);

    // Assert - weryfikacja rezultatu
    result.Should().BeOfType<OkObjectResult>();
}
```

3. Używanie Mocków

- Mockuj wszystkie zewnętrzne zależności
- Weryfikuj wywołania serwisów z `Times.Once`
- Testuj różne scenariusze błędów

4. Testy E2E

- Zawsze sprawdzaj dostępność serwera przed testem
- Czyść dane testowe po każdym teście
- Używaj unikalnych identyfikatorów dla zasobów testowych

Rozwiązywanie Problemów

Błąd: "The entry point exited without ever building an IHost"

- Problem z konfiguracją WebApplicationFactory
- Rozwiązanie: Użyj osobnej klasy testowej bez dziedziczenia z BaseTestClass

Błąd: "Type to mock must be an interface"

- Problem z mockowaniem sealed classes

- Rozwiązanie: Mock tylko interfejsy, nie konkretne klasy

Błąd: "Server not available" w testach E2E

- Serwer API nie jest uruchomiony
- Rozwiązanie: Uruchom `dotnet run` w katalogu `src/HyperV.Agent`

Przykłady Użycia

Dodawanie Nowego Testu

```
[Fact]
public async Task NewEndpoint_WithValidData_ShouldReturnCreatedResult()
{
    // Arrange
    var request = TestDataGenerator.CreateNewRequest();
    _mockService.Setup(x => x.CreateNew(request))
        .ReturnsAsync("created-id");

    // Act
    var result = await _controller.CreateNew(request);

    // Assert
    result.Should().BeOfType<CreatedResult>();
    var createdResult = (CreatedResult)result;
    createdResult.StatusCode.Should().Be(201);

    _mockService.Verify(x => x.CreateNew(request), Times.Once);
}
```

Test End-to-End

```
[Fact]
public async Task CompleteWorkflow_E2E_ShouldWork()
{
    // Test complete workflow against real API
    var createRequest = TestDataGenerator.CreateRequest();

    // Create
    var createResponse = await PostAsync("/api/v1/resource", createRequest);
    AssertStatusCode(createResponse, HttpStatusCode.Created);

    // Read
    var getResponse = await GetAsync("/api/v1/resource/test-id");
    AssertStatusCode(getResponse, HttpStatusCode.OK);

    // Update
    var updateResponse = await PutAsync("/api/v1/resource/test-id",
updateRequest);
```

```
    AssertStatusCode(updateResponse, HttpStatusCode.OK);

    // Delete
    var deleteResponse = await DeleteAsync("/api/v1/resource/test-id");
    AssertStatusCode(deleteResponse, HttpStatusCode.OK);
}
```

Następne Kroki

1. **Implementacja pozostałych kontrolerów**:
    - Skopiuj wzorzec z `ServiceControllerTests.cs` lub `JobsControllerTests.cs`
    - Dostosuj do specyfiki każdego kontrolera
    - Dodaj odpowiednie mocki do `ServiceMocks.cs`
2. **Rozszerzenie testów end-to-end**:
    - Dodaj testy dla kompleksowych scenariuszy
    - Implementuj cleanup danych testowych
    - Dodaj testy wydajnościowe
3. **Konfiguracja CI/CD**:
    - Integracja z systemem budowania
    - Automatyczne uruchamianie testów
    - Raportowanie pokrycia kodu

Wsparcie Techniczne

Jeśli napotkasz problemy:

1. Sprawdź, czy wszystkie zależności są zainstalowane
2. Upewnij się, że serwer API jest uruchomiony (dla testów E2E)
3. Sprawdź logi w terminalu podczas uruchamiania testów
4. Skonsultuj się z przykładami w istniejących klasach testowych